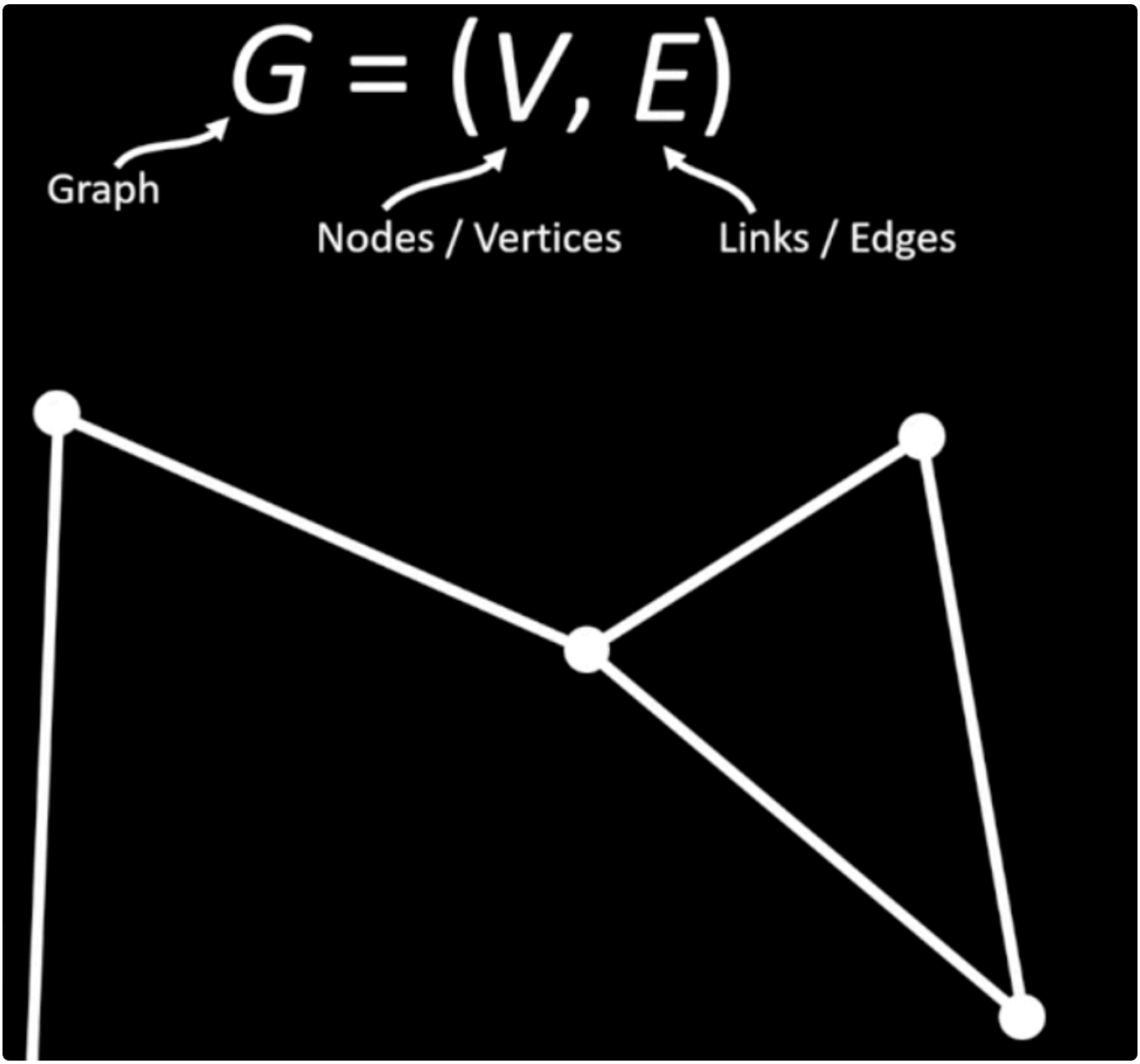
Self Driving Car - Project

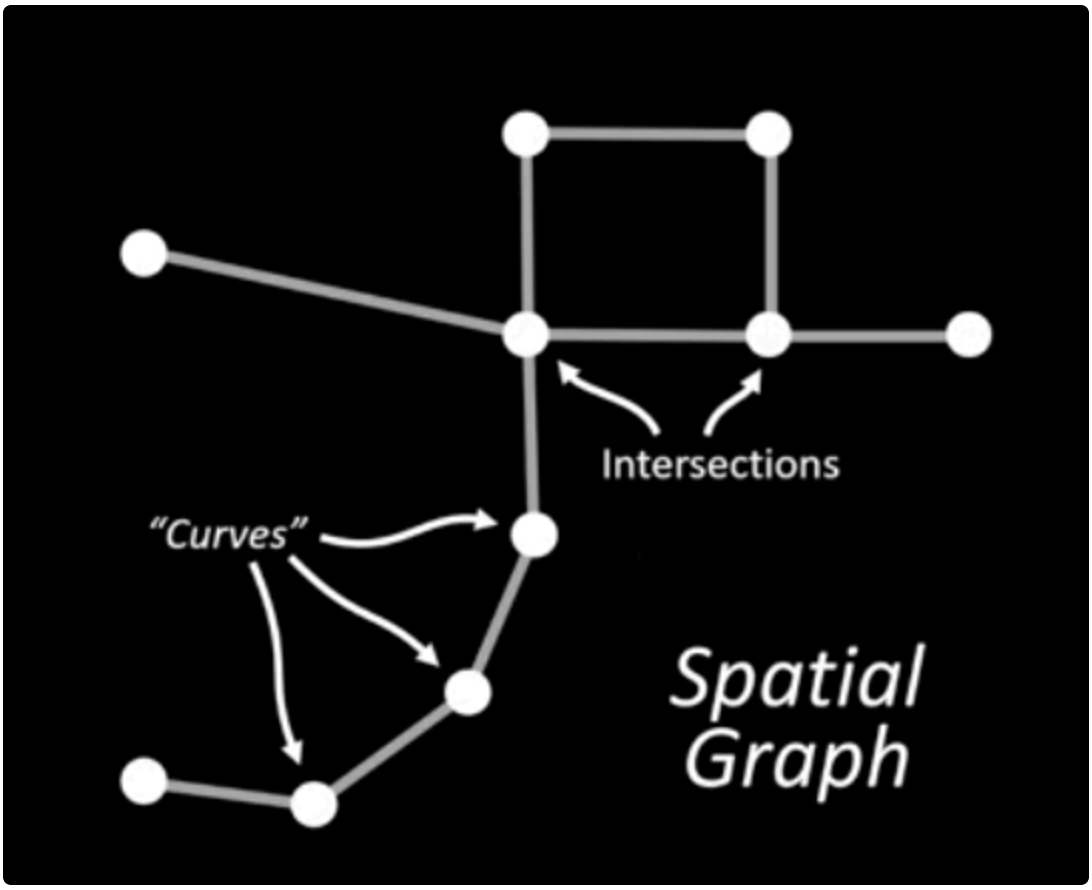
#Things_To_know

- Whatever you learn have the concept in how it used and when to use it and in what scenario we are using it and visualize how it help virtually.



data - openstreetmap


G = (V, E)
Graph
Nodes / Vertices
Links / Edges



When the central angle is measured in degrees, the arc length formula is:

$$\textbf{Arc length} = 2\pi r(\theta/360)$$

where,

θ indicates the central angle of the arc in degrees

r indicates the radius of the arc

Since, we know,

Circumference of the circle = $2\pi r$

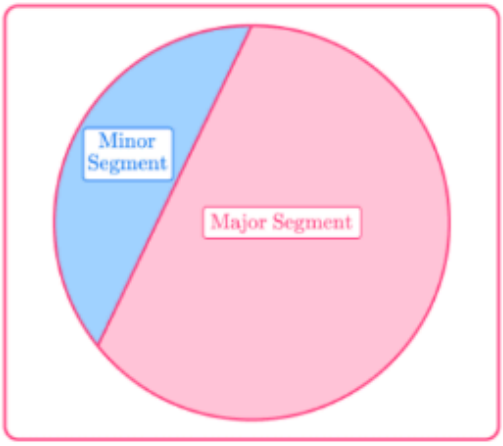
Therefore, length of the arc = $C(\theta/360°)$

Segment of a Circle

A **segment** of a circle, also called a circular segment, is the area enclosed by an arc of a circle and a chord.

There are two main types of segment:

- The **major segment** is the segment where the arc length is greater than half the circumference of the circle.
- The **minor segment** is the segment where the arc length is less than half the circumference of the circle.



script.js > ...

```
canvas2D.height = 600
canvas2D.width = 600

// graph
const ctx = canvas2D.getContext("2d");

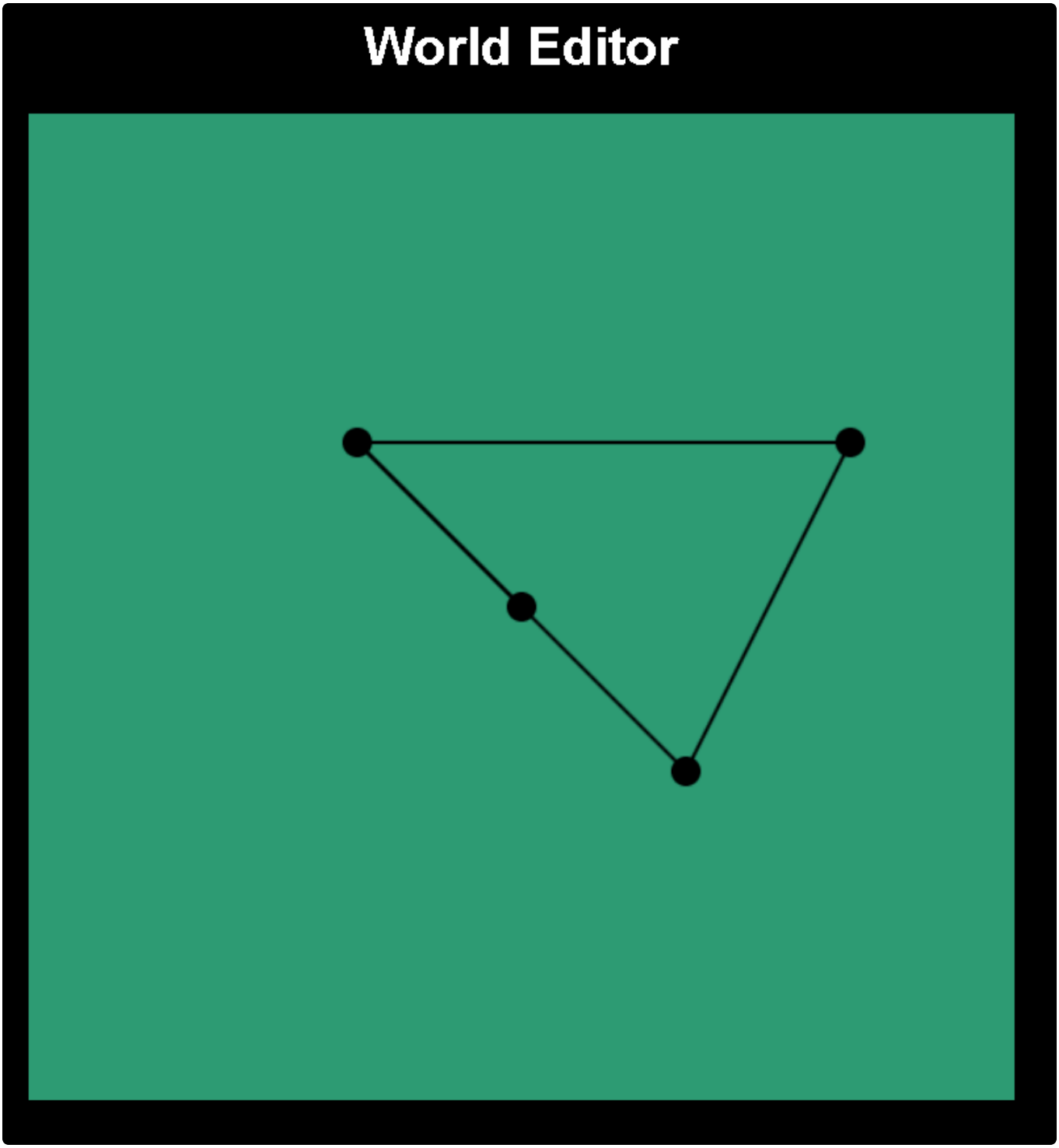
const p1 = new Point(200, 200);
const p2 = new Point(500 , 200);
const p3 = new Point(400 , 400);
const p4 = new Point(300 , 300);

const s1 = new Segment(p1 ,p2);
const s2 = new Segment(p3 ,p1);
const s3 = new Segment(p1 ,p4);
const s4 = new Segment(p2 ,p3);

const graph = new Graph([p1,p2,p3,p4] , [s1,s2,s3,s4]);
graph.draw(ctx);
```

this is a driver forming the canvas and also adding points and segement connecting those points

on a 2D graph.



from the top left of canvas(0,0) and (+x , +y) point on right x values and down y values and vice versa.



this is a point class having point created on the canvas or graph.

```
class Segment{
    constructor(p1 , p2){
        this.p1 = p1;
        this.p2 = p2;
    }

    draw(ctx , width = 2, color = "black"){
        ctx.beginPath();
        ctx.lineWidth  = width;
        ctx.strokeStyle  = color;
        ctx.moveTo(this.p1.x , this.p1.y);
        ctx.lineTo(this.p2.x , this.p2.y);
        ctx.stroke();
    }
}
```

and this is the segement class having stroke betweeen points

Now we will do the points adding dynamic and also the segement addition.

script.js > ...

```
canvas2D.height = 600;
canvas2D.width = 600;
// adding random point
function addRandomPoint() {
  graph.addPoint(
    new Point(Math.random() * canvas2D.width, Math.random()
  );
  ctx.clearRect(0, 0, canvas2D.width, canvas2D.height);
  graph.draw(ctx);
}
// graph
const ctx = canvas2D.getContext("2d");

const p1 = new Point(200, 200);
const p2 = new Point(500, 200);
const p3 = new Point(400, 400);
const p4 = new Point(300, 300);

const s1 = new Segment(p1, p2);
const s2 = new Segment(p3, p1);
const s3 = new Segment(p1, p4);
const s4 = new Segment(p2, p3);

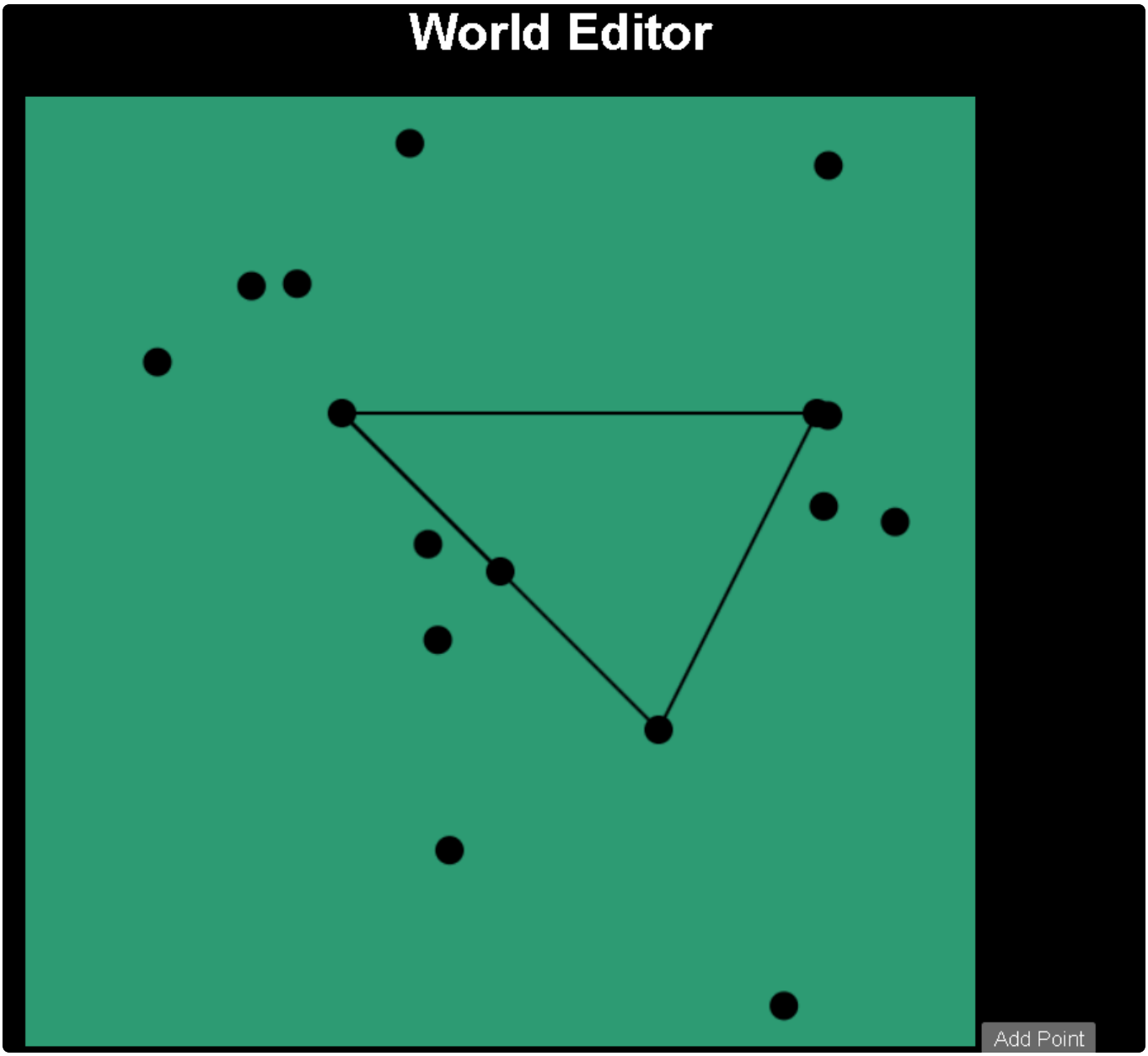
const graph = new Graph([p1, p2, p3, p4], [s1, s2, s3, s4]);
graph.draw(ctx);
```

a button is added in front and a event listener on click addRandomPoint is called that random function generates a new point x and y values and will clear the canvas and draw it again .

```
class Graph{
    constructor(points = [] ,segments =[]){
        this.points = points;
        this.segments = segments;
    }
    draw(ctx){
        for(const seg of this.segments){
            seg.draw(ctx);
        }

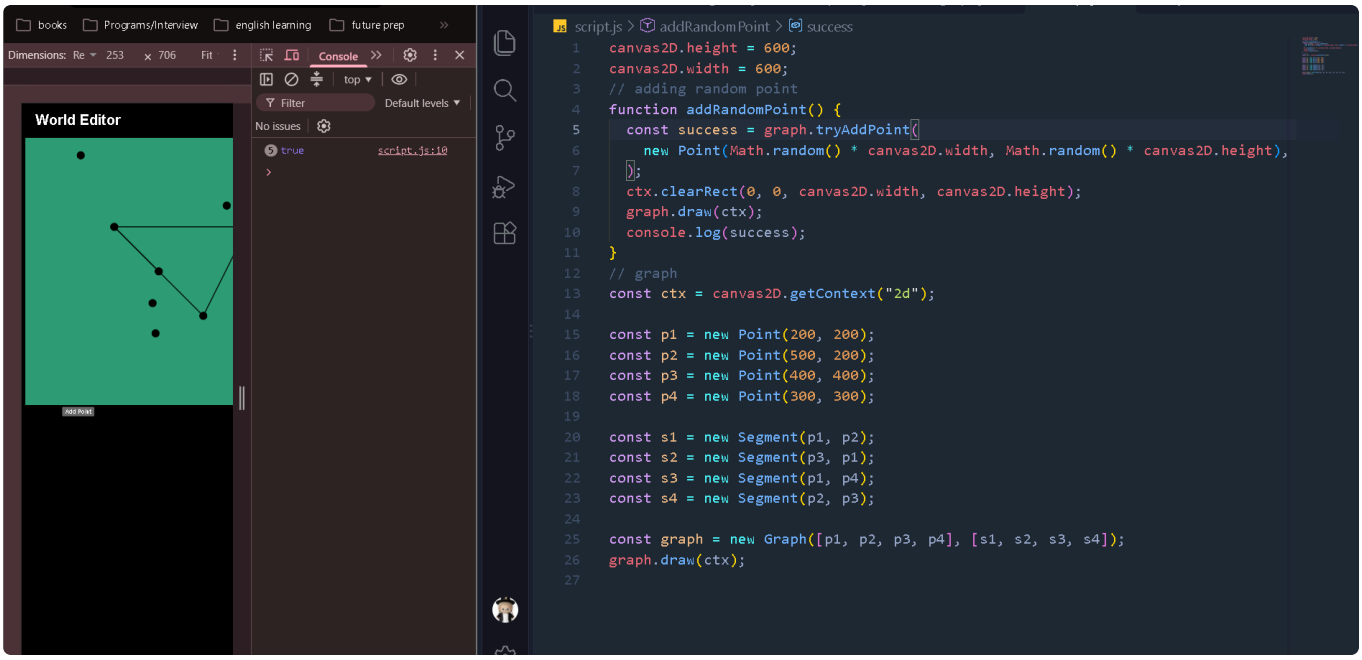
        for(const point of this.points){
            point.draw(ctx);
        }
    }

    addPoint(point){
        //  adding the random point
        this.points.push(point);
    }
}
```



Now #problem

- Adding two points at same location


books
Programs/Interview
english learning
future prep
Dimensions: Re 253 × 706 Fit
Console
top
Filter
Default levels
No issues
true
script.js:10
World Editor
Add Point
script.js > addRandomPoint > success
canvas2D.height = 600;
canvas2D.width = 600;
// adding random point
function addRandomPoint() {
const success = graph.tryAddPoint(
new Point(Math.random() * canvas2D.width, Math.random() * canvas2D.height),
);
ctx.clearRect(0, 0, canvas2D.width, canvas2D.height);
graph.draw(ctx);
console.log(success);
}
// graph
const ctx = canvas2D.getContext("2d");
const p1 = new Point(200, 200);
const p2 = new Point(500, 200);
const p3 = new Point(400, 400);
const p4 = new Point(300, 300);
const s1 = new Segment(p1, p2);
const s2 = new Segment(p3, p1);
const s3 = new Segment(p1, p4);
const s4 = new Segment(p2, p3);
const graph = new Graph([p1, p2, p3, p4], [s1, s2, s3, s4]);
graph.draw(ctx);



```
class Graph{
    draw(ctx){
    }

    addPoint(point){
        //  adding the random point
        this.points.push(point);
    }

    containPoint(point){
       for(let p of this.points){
        if(p.equals(point)){
            return true;
        }
       }
       return false;

    //    this.points.find((p) => p.equals(point))
    }

    tryAddPoint(point){
        if(!this.containPoint(point)){
            this.addPoint(point);
            return true;
        }
        return false;
    }


}
```

primitives > point.js > Point > equals

```
class Point{
    constructor(x,y){
        this.x = x;
        this.y = y;
    }

                                (parameter) color: string
    draw(ctx , size = 18 , color = "black"){
        const rad = size / 2;
        ctx.beginPath();
        ctx.fillStyle = color;
        ctx.arc(this.x , this.y , rad  ,0 , Math.PI * 2); //arc of a cricle
        // 360deg is 2pi radians
        ctx.fill();
    }

    equals(p){
        if(this.x === p.x && this.y === p.y){
            return true;
        }
        return false;
    }
}
```